IMPERIAL

VERITAS

MEng Individual Project

Imperial College London

Department of Computing

Tools for Foundational Verification of Probabilistic Programs

Author:
Edwin Fernando

Supervisor:
Shing Hin Ho
Dr. Azalea Raad

Second Marker:
Prof. Nicolas Wu

January 22, 2026

Contents

1 Introduction **3**
- 1.1 Objectives . . . 4
- 1.2 Challenges . . . 4
- 1.3 Contributions . . . 4

2 Background **5**
- 2.1 Measure Theory . . . 5
- 2.2 Semantics of Probabilistic Programs . . . 6
  - 2.2.1 Types in BPPL . . . 6
  - 2.2.2 Semantics of Let binding . . . 6
  - 2.2.3 Semantics of sample, observe and normalize . . . 7
- 2.3 Lean and relevant recent formalisations . . . 8
- 2.4 Separation Logic . . . 8
  - 2.4.1 Reviewing Probabilistic Separation Logics . . . 8
  - 2.4.2 Lilac . . . 9
- 2.5 Iris and MoSeL . . . 9
- 2.6 Related Work . . . 10

3 Project Plan **11**

4 Formalisation of Lilac **13**
- 4.1 Parametrisation of the ”store” by the ”heap” in this separation logic . . . 13
- 4.2 Pi-lambda-theorem as an induction principle . . . 14

5 Evaluation **15**

Chapter 1

Introduction

Probabilistic programming languages (Hakaru, Stan, Gen, etc [1–3]) offer a principled way of creating statistical models complex probability distributions over multiple random variables, as code. The programmer (statistician) can then interact with the model, evaluating (sampling from) the generated distribution. This decouples the specification of a model from the inference algorithm used for sampling. Probabilistic programming is in fact also pervasive in computer science, though generally not cleanly abstracted into a DSL. Examples include Zero Knowledge Proof Systems (Blockchain) [4], Differential Privacy and Machine Learning (TensorFlow Probability [5]). Formal reasoning about programs in these critical applications may be highly beneficial.

The aim of this project is to extend the reach of foundational verification in this direction. The main contributions are:

1. **C1**: Implementing (formalising) Lilac [6] and BaSL [7] separation logics in the Lean theorem prover, with "tactical support", using Iris [8].

2. **C2**: (if time permits) Develop a theory of compositional symbolic execution (CSE) for probabilistic programs and implement using Soteira [9]. CSE can automate program verification.

C1 is one of the first (if not the first) foundationally verified (in Lean) implementation of a probabilistic separation logic (PSL). There is in fact a simultaneous project formalising the Bluebell PSL [10] for the purpose of verifying the core of a Zero Knowledge Proof system [4].

C2 would be a novel theoretical contribution. The main innovation here, would be either The separation logic chosen for this may be Lilac or BaSL, but also possibly Bluebell [11], to directly apply the work in an industrial formalisation project. However Bluebell supports both unary

with various subsystems But there are much wider applications in computer science, where arguments of correctness are probabilistic in nature. Examples expressively and formally verify these arguments if the implementation was carried out in a probabilistic programming language/DSL, using Probabilistic Separation Logics (PSL). This project is meant as a proof of concept, aiming to be the first implementation of a PSL. Specifically in the Lean proof assistant, we implement the lilac separation logic using Iris-lean (ported from rocq) to provide ergonomic tactical support. We choose Lean for the significant body of measure-theoretic results present in Mathlib, required to establish soundness of the logic.

Given time I would also like to pursue formalisation of BaSL or a theory for symbolic execution using lilac or BaSL for probabilistic programs.

on top of the Iris framework, in Lean. offer an intuitive way to reason about probabilistic programs at a suitable level of abstraction.

1.1 Objectives

1.2 Challenges

1.3 Contributions

Chapter 2

Background

Probabilistic programming is a paradigm where we are defining statistical models using code. It is a means of describing complex distributions over some space in a structured way. Probabilistic programming language (PPL) also come with an interpreter (probabilistic inference engine) ehich allows the programmer to sample a term from the distribution denoted by a program and indeed sample distributions while constructing new distributions.

More interestingly a Bayesian probabilistic programming language (BPPL) also allows one to "observe" an event and update their belief of the world (update the probability distribution they associate with some ocurrence). Here we take the bayesian perspective underpinned by Baye's law which describes how we update a probability distribution based

TODO: buses and days and instantiation of Baye's law

A simple example taken from [12] is reproduced below to show how

2.1 Measure Theory

In order to understand the semantics of a probabilistic programming, a formal definition for probability is needed. Measure theory underpins probability theory and an overview is given here. We want to be able to talk about the probabilities of some event occurring, which is modelled by a `random variable`. In order to define a random variable we first need to define Measures and Measurable Space. A measurable space is a pair $(\Omega, \mathcal{F})$, where Ω is some Set, and $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ is called the σ-algebra. We interpret each element of $\mathcal{F}$ to be an event (such as a coin flip landing heads). We would like the σ-algebra to closed under complements and countable unions. From the presentation so far it may seem like all singleton sets must be events but due to technical reasons (see [13] Ch. 2), this cannot be in certain cases where Σ is infinite.

A measure on a measurable space is a function $\mu : \mathcal{F} \to [0, \infty]$

This is essentially providing a probability density or probability mass function for the set of events $\mathcal{F}$. A similar but more general notion than what a measure gives us, is a random variable: a measurable function between two measurable spaces. I say "more general notion", since using the identity measurable function yields the measure we started with. A measurable function denoted $f : (\Omega_1, \mathcal{F}) \xrightarrow{m} (\Omega_2, \mathcal{G})$ $f : \Omega_1 \xrightarrow{m} \Omega_2$ is well defined when there is an ambient measurable space structure $(\Omega_1, \mathcal{F})$ and $(\Omega_2, \mathcal{G})$. f allows us to talk about the distribution of a particular attribute of in the event space. For example $\Omega_1 = [1, 6]^2$ may be the set of all events associated with playing 2 dice, while f may map to the sum of the two dice, $\Omega_2 = [1, 12]^2$. A measurable function f, transforms any measure μ on $(\Omega_1, \mathcal{F})$ to the pushforward measure μ' on $(\Omega_2, \mathcal{G})$. In order for this to always be well defined we require the following measurability to be satisfied by all measurable functions: $\forall G \in \mathcal{G}, f^{-1}(G) \in \mathcal{F}$.

TODO: Define lebesgue integration, since it's used in the Giry monad. TODO: Define s-finite measures and kernels

2.2 Semantics of Probabilistic Programs

Staton provided a compositional semantics to first-order Bayesian probabilistic programs in [12], which forms the foundation of program logics, validation of inference algorithms, etc. The scope of this project is entirely within first-order probabilistic programs (probability distributions over function spaces are not expressible). The programming language considered in Lilac uses a simpler semantics since it doesn't support bayesian updates. BaSL uses exactly this semantics. Note that supporting bayesian updates, involves the construct `observe` and `normalize` in the language. We will present here the more complete semantics. The key contribution of the paper is the introduction of `s-finite kernels`, motivated by validating a basic commutativity property that programmers expect. Specifically: when neither x is free in u nor y is free in t. Another benefit of

$$\begin{array}{l} \texttt{let } x = t \texttt{ in} \\ \texttt{let } y = u \texttt{ in} \\ v \end{array} \quad = \quad \begin{array}{l} \texttt{let } y = u \texttt{ in} \\ \texttt{let } x = t \texttt{ in} \\ v \end{array}$$

Figure 2.1: commutative program transformation

this semantics is a line-by-line compositionality, ie, the semantics of every subterm in a program is well-defined, not just whole programs. This is just another way of saying the semantics of open programs (those with free variables), as well as closed programs is well-defined, respectively. Validating proof rules in a logic is much better formulated in terms of such a compositional semantics. s-finite kernels denote open programs and s-finite measures denote closed programs.

2.2.1 Types in BPPL

The setting is the category of Measurable Spaces, `Meas`. All types in our language is interpreted as an object in this category. Syntactically:

$$\mathbb{A}, \mathbb{B} ::= \mathbb{R} \mid \mathcal{P}(\mathbb{A}) \mid 1 \mid \mathbb{A} \times \mathbb{B} \mid \cdots$$

Objects in the category of the form (A, Σ_A). Morphisms are measurable functions. The interesting type here is $\mathcal{P}(\mathbb{A})$. It's the type of probability distributions over $\mathbb{A}$. Actually not quite, since we allow any measure $\mu : \Sigma_A \to [0, \infty]$, rather than $\Sigma_A \to [0, 1]$. Now in order to be an object in the category, $\mathcal{P}(\mathbb{A})$, the set of all measures on $\mathbb{A}$. must itself form a measurable space. There is some construction to form a σ-algebra for $\mathcal{P}(\mathbb{A})$ satisfying all the requirements (we don't go into it here).

2.2.2 Semantics of Let binding

Probabilistic programs are interpreted using the Giry monad [14] in the category of Measurable Space, `Meas`. Probability is an effect. This interpretation gives us the semantics of sequencing (let-binding) so we go through it below. Kernels also naturally show up in type of the monadic bind, which motivates it.

The Giry functor takes some object (measurable space) (A, Σ_A) to a measurable space on the measures of $(\Sigma_A \to [0, \infty], \cdots)$ (the σ-algebra is ommitted). We'll denote $\Sigma_A \to [0, \infty]$ as `Measure A`, for readability.

The explanation focuses on intuition through diagrams. For a measurabe space (A, Σ_A), a probability measure is of type $\mu : \Sigma_A \to [0, 1]$. This gives a distribution on A. The intuition behind μ is that we can sample some event using it. A kernel from A to B has type $\kappa : A \to MeasureB$. Each triangle below is a kernel. And the types of it input and output are indicated, and we compose the kernels along the type B as $\kappa \underset{B}{\circ} \iota$. The types of the induvidual kernels and composed kernel of Fig 2.2.2, is given below

$$\iota : A \to \text{Measure } B \tag{2.1}$$
$$\kappa : B \to \text{Measure } C \tag{2.2}$$
$$\kappa \underset{B}{\circ} \iota : A \to \text{Measure } C \tag{2.3}$$

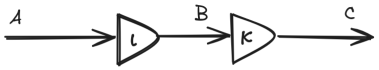


Figure 2.2: A simple probabilistic program

The interpretation of the kernel ι is that for every (deterministic) value of A it gets, it will give you a (probabilistic value) distribution over type B. So composition here, is sampling from the `Measure` B, getting a deterministic value of type B and passing that to the kernel κ. This sampling is done ranging over all the values of B in proportion to their distribution, and taking a weighted average of the values of type `Measure` C. More precisely, this is the integral: $\kappa \underset{B}{\circ} \iota = \lambda a.\, \int \lambda b.\, \kappa(b)\, d\iota(a)$ (integral will be explained in 2.1 in the final draft). Notice that $\circ$ has a type very similar to the monadic bind (in the Giry monad). We now write $\mathcal{G}(A)$ instead of `Measure` A, for the Giry functor. $\circ : ((B) \to \mathcal{G}(C)) \to ((A) \to \mathcal{G}(B)) \to \mathcal{G}(C)$. If we just swap the arguments and partially apply an $a : A$ on ι, we see that the bind of the Giry monad composes a measure with a kernel like so $\iota(a) \gg\!= \kappa : \mathcal{G}(B) \to (B \to \mathcal{G}(C)) \to \mathcal{G}(C)$

The most interesting part of Staton's commutative semantics is that when kernels have multiple inputs (this does extend into the realm of multi-categories but we don't go into that here). To clarify, these kernels have multiple deterministic values they take in order to return a single probabilistic value (a measure). The commutativity property illustrated by the equality of the two programs in Fig 2.2.2.

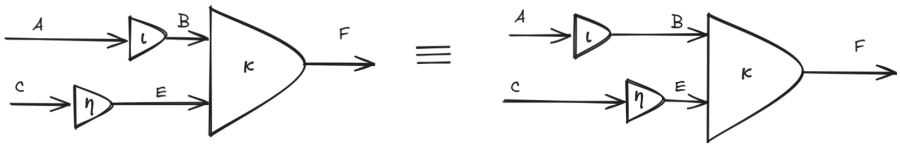


Figure 2.3: Commutativity of let bindings

$\kappa \underset{B}{\circ} \iota$ is given by the following on the left and right respectively

$\lambda a\ c.\, \int \lambda e.\, \left(\int \lambda b.\, \kappa(b, e)\, d\iota(a)\right) d\eta(c)$

$\lambda a\ c.\, \int \lambda b.\, \left(\int \lambda e.\, \kappa(b, e)\, d\eta(c)\right) d\iota(a)$

Though these look rather unreadable, all that's happening is swapping the order of integration, and there is a theorem which allows us to this in certain conditions. In [12] it is shown that this theorem holds if instead of general kernels, we restrict ourselves to s-finite-kernels. A kernel $\kappa : A \to \mathcal{G}(B)$ is finite if there is finite $r \in [0, \infty)$ such that, for all a, $\kappa(a, B) < r$. κ is s-finite if there is a sequence $\kappa_1 \ \ldots \ \kappa_n \ \ldots$ of finite kernels and $\Sigma_{i=1\ldots\infty}\, \kappa_i = \kappa$.

In short topological transformations of these diagrams corresponding to the motivating example in 2.1 do not change the semantics.

2.2.3 Semantics of sample, observe and normalize

The bayesian probabilistic programming language (BPPL) is a standard ML like language with 3 special contstructs `sample`, `observe` and `normalize`

TODO: a description of these constructs

2.3 Lean and relevant recent formalisations

Foundational verification, consists of machine checked proofs of theorems, assuming only the axioms of mathematics. Here we are concerned with verification of software, so the theorems in question express properties of the software. The axioms of mathematics that we use is a dependent type theory. In this setting of dependent type theory, checking the correctness of proof is the same thing as checking if a proof (program) is well typed, due to the Curry-Howard Isomorphism CITE. Further the tool we use to for machine-checked proofs is an interactive theorem prover with support for proofs using "tactics". Tactical support is just metaprogramming, trying to encapsulate reasoning principles a person would use when writing pen-and-paper proofs. There are two proof assistants that were strongly considered for the purposes of this project: Lean and Rocq, former having proofs of required theorems in measure theory and the latter having the original implementation of the Iris framework. Lean was chosen since the front-end of the Iris proof interface (MoSeL, see 2.5) had been fully ported to Lean very recently.

There has been an exciting amount of recent activity in formalisations related to this work, which have enabled the formalisation of Lilac (see, **??**) in Lean to be a realistic goal

A lot of background in Lean was required to undertake this project. The Lean development exercises the full expressivity of inductive types in Lean, for example and a thorough understanding of propositional vs definitional equality which is related to Proof irrelevance [Theorem proving for Lean4] and how that's implemented in Lean's type theory by special treatment of the lowest Universe [Mario's thesis]

2.4 Separation Logic

Separation logic has been extremely productive in the last two decades in reasoning about imperative programming languages, after first being introduced in [15]. The classical separation logic introduced a compositional state, specifically a heap (modelled as a function $\mathbb{N} \to Val$), which was required along with a more usual variable environment, to interpret any assertion. This heap or more generally resource, had an algebraic structure to it, the most fundamental one being a partial commutative monoid (PCM) with the binary operator dictating when it is valid to combine to heaps. Many more complex algebras for resources were studied in later works [16, 17]. Separation logic was identified as a very natural tool reason about imperative programs mutating heap based memory, and concurrent programs.

2.4.1 Reviewing Probabilistic Separation Logics

More recently there has been a lot of research into probabilistic separation logics: [18], introduced separation as probabilistic independence. [6] introduced Lilac, which allowed usage of continuous distributions, providing a measure theoretic definition of resources. [7] introduced reasoning about bayesian updates (supporting the `observe` and `normalize` constructs), thereby providing an axiomatic semantics for the general form of first order probabilistic programs (ie, BPPL as introduced by [12]).

There is a parallel line of work in relational separation logics. One could argue that relational logics are more important in the probabilistic domain, since in bayesian probabilistic programing (so including `observe` and `normalize`), frequently generates probability distributions which have no closed form solution! Sampling from these can only be done using approximation methods like Variational Inference. The justification for relational logics classical is that relational properties are expressed more naturally, but could perhaps still be done using unary reasoning, in an inelegant way. In probabilistic programs however, relational reasoning might in fact be more expressive making it important. Relational reasoning for probabilistic programs (though not using a separation logic) can be found here [19]. More recently there has been a novel combination of relational and unary reasoning for probabilistic programs in a separation logic by [11].

2.4.2 Lilac

2.5 Iris and MoSeL

Iris [8] is a step-indexed modal separation logic partly designed as a framework for embedding other separation logics into, to take advantage of the Iris Proof Mode (IPM) [20]. The Iris proof mode provides state of the art "tactical support", tactic based interactive theorem proving. More recently the front of the IPM was abstracted from Iris to any separation logic to create MoSeL [21]. We aim to use MoSeL in this project for several reasons

1. Neither Lilac nor BaSL is step-indexed, so it is much cleaner to instantiate MoSeL rather than Iris
2. Making Lilac compatible with Lilac's interface (instantiating an algebraic structure called CMRA [8]) is an unsolved problem (future work section [6])
3. MoSeL was completely ported to Lean, shortly before the start of this project! [22].

MoSeL presents the interface of a "logic of Bunched Implications" which essentially contains all standard connectives of a first order logic and the separation logic connectives. The only addition MoSeL adds to this is a persistence modality (which can be instantiated in a trivial way in exchange for defeating any automation it might achieve?).

One of the key generalisations that MoSeL makes over Iris is parametricity over linear and affine logics. Iris is an affine logic meaning separating conjuncts in a premise can be used 0 or 1 time in proving a goal; they can be thrown away if desired. In a linear logic however every conjunct must be used exactly once, as in the first separation logic by O'Hearn and Pym [15]. There is naturally quite an emphasis on describing whether a logic is affine or linear, since throwing away unwanted predicates when it is suitable to do so, is exactly the kind of thing a user wants automation for. In BaSL [7], there is a short discussion stating BaSL is part affine and part linear. Precisely characterising the constructs which are affine and linear or under what conditions should be explored for better tactical support.

MoSeL like IPM before it provides tactics mirroring the native Lean tactics, such as `iIntros` and `IAssumption`

In an ongoing project to port Iris from Rocq to Lean, MoSeL was completely ported to Lean right before the beginning of this project [22, 23]. So the formalisation builds on this. The interface provided by MoSeL is quite general. The interface defines a pre-orderered Set (PROP, Entails)

```
class BIBase (PROP : Type u) where
%   ⊢ : PROP → PROP → Prop -- Entails
  pure : Prop → PROP
  ∧ : PROP → PROP → PROP
  → : PROP → PROP → PROP
  * : PROP → PROP → PROP -- separating conjunct
  -* : PROP → PROP → PROP -- magic wand
  persistently : PROP → PROP
  ...
```

The type class `BIBase` defines the interface for the "data" parts of the separation logic, while the axioms that these need to satisfy (the "proof" parts) are defined in an extension of this type class. We are defining the separation logic, Lilac or BaSL, in the meta-logic, Lean. So `BIBase` is parametrised by a type `PROP` which is a proposition in the separation logic (as opposed to `Prop` in lean).

We draw the reader's attention to the `Entails` which allows us to provide the semantics of entailment. The soundness of MoSeL's syntactic entailments are established by the semantics provided when instantiating `Entails`. This gives us a little flexibility in how we'd like to instantiate `PROP`. For simpler logics we could have `PROP`, directly capture the semantis of that proposition, giving `PROP` this shape.

```
PROP := Store → Heap → Prop
```

My understanding is that this precisely encodes a relation: the satisfiability relation. An n-ary relation is encoded as a function from the n parameters to a Proposition. In other words it's a set

of n-tuples. (Sets of type A are encoded as $A \to Prop$). Notice however that $s, h \vDash P$ should be of the form

```
PROP := Store → Heap → PROP → Prop
```

where did the P (of type `PROP`) go? It's given implicitly through the inductive construction of a term of `PROP` through the constructors defined by `and`, `sep`, etc. Any term of `PROP` implicitly contains the tree of constructors used to create it.

This is standard for the simple example instantiations of MoSeL (such as classical separation logic, but quite convoluted conceptuall. We may wish to split the representations of the separation logic assertions into syntax and semantics, which is what is done in non-trivial logics and this projects formalisation. There is a denotation function of type $Store \to Heap \to PROP \to Prop$ (exactly how the satisfiability relation should look) used in the instantiation of Entails; much more natural!

2.6 Related Work

TODO: List the quasi borel spaces formalisation and stuff

Chapter 3

Project Plan

I want to note that I have not before seen the interpretation of a typed calculus, taking both the type and typing context as parameters to give the denotation of a term. This definition of terms with they're typing context, actually massively simplifies the lean development previous iterations, involved a type inference algorithm, which could fail and the denotation function was partial.

I present 3 possible goals for the project here. `M1` is the fallback which is mostly formalisation of already existing theory. `M2` and `M3` are more ambitious goals involving novel theoretical contributions. One of my original motivations for a formalisation project was to thoroughly learn and understand a topic in programming language theory, all the way up to the state of the art. Simultaneously I would be making a non-trivial contribution by formalising a program logic in a theorem prover and developing "tactical support" for it, showing a prototype for what is possible using foundational verification at the moment. On top of formally verifying the soundness of Lilac and BaSL, this could show how ergonomic or expressive a logic is. However this is still only a prototype because of the requirement to use the toy language (BPPL) This is only a "prototype"

The original scope of the project was `M1` alone but t

In the case of `M2` there is also a scope of industrial use of an implementation in Lean.

- Formalisation of Lilac with tactical support
- Lean interpreter for the language which is sound w.r.t to the denotational semantics
- The first case of Compositional Symbolic execution for probabilistic separation logic

  CSE for the Bluebell separation logic with possible immediate application in industry for verifying correctness of Zero Knowledge Proof systems This could perhaps be a tactic, which actually invokes the symbolic execution to elaborate into a sequence of proof rules?

  This is possibly not so hard

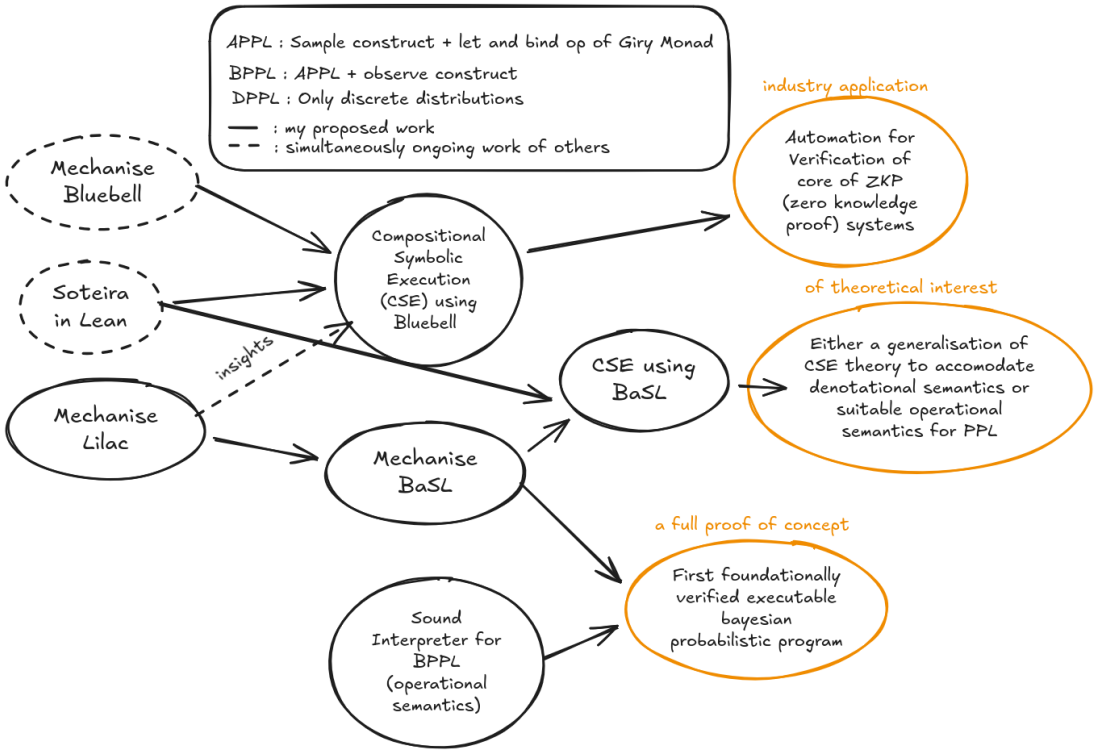
  CSE for Lilac or BaSL. this is perhaps out of scope of project. But it connects to the first part of the project formalising Lilac/BaSL which is novel for being the first mechanisation of a separation logic for continuous probabilistic programs.

Put simply my goal is to reach one of the orange boxes. The "full proof of concept" is fallback goal. The major contribution here would be the formalisation of BaSL in Lean, which has several unknown elements of pure math proofs to it, when compared to to the Lilac formalisation. To name a few 1) Pi-lambda theorem (elaborate) 2) Disintegration theorem

The goal "industrial application" requires the development of a theory of compositional symbolic execution using Bluebell as the assertion language. So even if the Lean formalisations I am dependent on to implement a tool, using the theory is not available in time, I can still make a significant theoretical contribution here.

is dependent on completion of work planned by others, by the time I reach that stage of my project.

Figure 3.1: Example of a parametric plot $(\sin(x), \cos(x), x)$



Chapter 4

Formalisation of Lilac

Here I present the current progress in the formalisation of Lilac, and how I went about it. I divide it into 2 parts Initial work which was mostly measure theoretic, and recent work which is more structured. As in any project its best to get to the most uncertain parts of it first, to minimise its risk. I think I have somewhat intuitively followed this principle in my formalisation so far, also meaning its full of `sorrys`. However the shape of the formalisation of Lilac is reasonably clear and I was able to identify some noteworthy problems which show up in the formalisation.

The repository can be found on GitHub: It contains most of the syntax and semantics of both the language and program logic for it. These can be regarded as the "data" parts (as opposed to "proof" parts) of the formalisation. There was much consideration (and confusion) in modelling both language and logic in a principled manner. We want a representations for free variables and contexts where we don't go out of bounds by construction. The approach is inspired by Dependent de Bruijn Indices [24] which also shows how types/propositions and terms in the language/logic are translated consistently into lean types and terms repsectively when taking the denotation.

The goal with defining these, data parts is to be able to, using MoSeL, reimplement the case studies provided in the Lilac paper in an ergonomic manner (with tactical support), with the proofs of soundness of the logic itself being left as `sorry`. The "proof" parts are mostly incomplete, with the proof of unitarity of the unital element of the KRM being almost complete, being a first milestone (as it involves proof of corollaries of the π-λ-theorem)

As a brief reflection on the difference in formalisation on the maths and computer science sides, in my limited experience, I do think in this project (computer science) I do need to think of multiple things at once and papers generally disseminate dependencies more clearly without depending as heavily on the reader to morph and modify lemmas as required. However the objects in Lilac are so unconventional from the measure theory in mathlib, that it requires inventing some things from scratch and quite challenging. This is ideal for my goals of understanding how to use a dependent type theory for formalisation. As such 4.2 talking about formalisation specific issues. But 4.1 talks about something more conceptual in Lilac! (My pivot towards the aim of a more theoretical contribution is that I feel I have gotten enough Knowledge of Lean from formalising Lilac)

4.1 Parametrisation of the "store" by the "heap" in this separation logic

The classical separation logic assertions are written like so $s, h \vDash P$, where the s is a variable store and h is a heap. In the satisfiability relation, none of s, h or P are dependent on another argument. Consider the probabilistic separation logic with satisfiability relation of form $\gamma, D, \mathcal{P} \vDash P$, where γ is the deterministic variable store, D is the random variable store and $\mathcal{P}$ is the probability space analagous to heap. I claim that D is dependent on $\mathcal{P}$, and that many of the conditions in the satisfiability relation are actually true by construction.

This unspotted dependent typing is a consequence of how measurable functions are usually just denoted $A \xrightarrow{m} B$ with the σ-algebra on A and B that is required being generally obvious from the

context (there is usually only one). However in our case, we our changing the *sigma*-algebra we're working with all the time when taking independent combinations, in the separating conjunct.

The relevant measurable function here is D the random variable store. Its type is $D : [0,1]^{\mathbb{N}} \xrightarrow{m} [\![\Delta]\!]$, where $[\![\Delta]\!]$ is the denotation of the random variable context, a list of types the random values that can occur in the separation logic proposition P. As seen in a PSL, D has quite an interesting type as a measurable function which can be composed with an expression in a propostion, which is equivalent to substitution for the usual deterministic variables. Now no explicit σ-algebra is provided for the Hilbert cube, the domain of D and the natural candidate for this is the *sigma*-algebra for the hilbert cube contained in the probability space $\mathcal{P}$. This interpretation yeilds some simplifications in the satisfiability relation of probability specific connectives, exemplified in the `Own` connective. It's semantics is:

$\gamma, , ([0,1]^{\mathbb{N}}, F, \mu) \vDash own \quad iff \quad (\gamma) \circ$ is F-measurable where $\mathcal{P} = ([0,1]^{\mathbb{N}}, F, \mu)$ and F is $\mathcal{P}$'s σ-algebra. So $(\gamma) \circ$ is F-measurable by construction. Ownership as measurability is a key concept of Lilac, and it neatly falls into place.

4.2 Pi-lambda-theorem as an induction principle

The π-λ-theorem is tool in measure theory which can be used for equality of two measures. We use it prove a certain uniqueness of measures theorem, used to establish that separating conjunction yeilds a PCM (partial commutative monoid) structure. It is similar to an induction principle on the structure of a σ-algebra. So we need prove some property C holds in the base and inductive cases of a σ-algebra. However there isn't an inductive type underlying this induction principle! The reason is two fold 1) when generating a measurable set from some base measurable sets, a specific set can be shown to be measurable by multiple different permutations of application of the axioms. For reference below is included an inductive defintion generating a σ-algebra from base measurable seats.

```
/-- The smallest σ-algebra containing a collection `s` of basic sets -/
inductive GenerateMeasurable (s : Set (Set α)) : Set α → Prop
  | protected basic : ∀ u ∈ s, GenerateMeasurable s u
  | protected empty : GenerateMeasurable s ∅
  | protected compl : ∀ t, GenerateMeasurable s t → GenerateMeasurable s tᶜ
  | protected iUnion : ∀ f : ℕ → Set α, (∀ n, GenerateMeasurable s (f n)) →
      GenerateMeasurable s (⋃ i, f i)
```

2) The π-λ-theorem provides a relaxation on our proof obligation. We dont prove that the property C holds for countable unions (`iUnion` above) if it holds for each of the sets we're taking a union over. Instead we only need to do so for countably pairwise disjoint unions.

Being closed under disjoint unions instead of unions, is the difference between a λ-system compared to a σ-algebra. This relaxtion is the content of the π-λ-theorem which we don't go into here. However there is an alternative "inductive" definition of a λ system with the following axioms:

1. The system $\mathcal{C}$ containt the base measurable sets we chose
2. be closed under proper difference, for $\forall V, U \in \mathcal{C}, V \subseteq U \to U \setminus V \in \mathcal{C}$
3. closed under increasing limits, take $i \in \mathbb{N}$ and $U_i \in \mathcal{C}$ then $\bigcup_{i \in \mathbb{N}} U_i \in \mathcal{C}$

Unfortunately this is not the definition of λ-system in Mathlib, and it doesn't provide the "induction principle" according to this definition. For the case of probability measures (μ (universal set) = 1), there is a simple workaround, which is sufficient for Lilac. However BaSL uses unnormalised measures since it support bayesian conditioning, and so, there may be no simple workaround but reproving this alternative induction principle.

Chapter 5

Evaluation

Bibliography

[1] Praveen Narayanan, Jacques Carette, Wren Romano, Chung-chieh Shan, and Robert Zinkov. Probabilistic inference by program transformation in hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings*, pages 62–79. Springer, 2016. doi: 10.1007/978-3-319-29604-3_5. URL http://dx.doi.org/10.1007/978-3-319-29604-3_5.

[2] Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1):1–32, 2017. doi: 10.18637/jss.v076.i01. URL https://www.jstatsoft.org/index.php/jss/article/view/v076i01.

[3] Marco F. Cusumano-Towner, Feras A. Saad, Alexander K. Lew, and Vikash K. Mansinghka. Gen: a general-purpose probabilistic programming system with programmable inference. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2019, page 221–236, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367127. doi: 10.1145/3314221.3314642. URL https://doi.org/10.1145/3314221.3314642.

[4] Verified-zkEVM. Arklib: Formally verified arguments of knowledge in lean, 2025. URL https://github.com/Verified-zkEVM/ArkLib. Accessed: 2025-12-15.

[5] Joshua V. Dillon, Ian Langmore, Dustin Tran, Eugene Brevdo, Srinivas Vasudevan, Dave Moore, Brian Patton, Alex Alemi, Matt Hoffman, and Rif A. Saurous. Tensorflow distributions, 2017. URL https://arxiv.org/abs/1711.10604.

[6] John M. Li, Amal Ahmed, and Steven Holtzen. Lilac: A modal separation logic for conditional probability. *Proc. ACM Program. Lang.*, 7(PLDI), June 2023. doi: 10.1145/3591226. URL https://doi.org/10.1145/3591226.

[7] Shing Hin Ho, Nicolas Wu, and Azalea Raad. Bayesian separation logic, 2025. URL https://arxiv.org/abs/2507.15530.

[8] Ralf Jung, Robbert Krebbers, Jacques-henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi: 10.1017/S0956796818000151.

[9] Sacha-Élie Ayoun, Opale Sjöstedt, and Azalea Raad. Soteria: Efficient symbolic execution as a functional library, 2025. URL https://arxiv.org/abs/2511.08729.

[10] Verified-zkEVM. iris-lean. https://github.com/Verified-zkEVM/iris-lean, 2025. Fork of the Iris Lean repository, instantiating a probabilistic separation logic, Bluebell.

[11] Jialu Bao, Emanuele D'Osualdo, and Azadeh Farzan. Bluebell: An alliance of relational lifting and independence for probabilistic reasoning. *Proc. ACM Program. Lang.*, 9(POPL), January 2025. doi: 10.1145/3704894. URL https://doi.org/10.1145/3704894.

[12] Sam Staton. Commutative semantics for probabilistic programming. In *Programming Languages and Systems: 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings*, page 855–879, Berlin, Heidelberg,

2017. Springer-Verlag. ISBN 978-3-662-54433-4. doi: 10.1007/978-3-662-54434-1_32. URL https://doi.org/10.1007/978-3-662-54434-1_32.

[13] Sheldon Axler. *Measures*, pages 13–72. Springer International Publishing, Cham, 2020. ISBN 978-3-030-33143-6. doi: 10.1007/978-3-030-33143-6_2. URL https://doi.org/10.1007/978-3-030-33143-6_2.

[14] Michèle Giry. A categorical approach to probability theory. In B. Banaschewski, editor, *Categorical Aspects of Topology and Analysis*, pages 68–85, Berlin, Heidelberg, 1982. Springer Berlin Heidelberg. ISBN 978-3-540-39041-1.

[15] Peter W. O'Hearn and David J. Pym. The logic of bunched implications. *Bulletin of Symbolic Logic*, 5(2):215–244, 1999. doi: 10.2307/421090.

[16] Cristiano Calcagno, Peter W. O'Hearn, and Hongseok Yang. Local action and abstract separation logic. In *22nd Annual IEEE Symposium on Logic in Computer Science (LICS 2007)*, pages 366–378, 2007. doi: 10.1109/LICS.2007.30.

[17] David J. Pym, Peter W. O'Hearn, and Hongseok Yang. Possible worlds and resources: the semantics of bi. *Theoretical Computer Science*, 315(1):257–305, 2004. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2003.11.020. URL https://www.sciencedirect.com/science/article/pii/S0304397503006248. Mathematical Foundations of Programming Semantics.

[18] Gilles Barthe, Justin Hsu, and Kevin Liao. A probabilistic separation logic. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–30, December 2019. ISSN 2475-1421. doi: 10.1145/3371123. URL http://dx.doi.org/10.1145/3371123.

[19] Gilles Barthe, Benjamin Grégoire, and Santiago Zanella Béguelin. Formal certification of code-based cryptographic proofs. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '09, page 90–101, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605583792. doi: 10.1145/1480881.1480894. URL https://doi.org/10.1145/1480881.1480894.

[20] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. *SIGPLAN Not.*, 52(1):205–217, January 2017. ISSN 0362-1340. doi: 10.1145/3093333.3009855. URL https://doi.org/10.1145/3093333.3009855.

[21] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. Mosel: a general, extensible modal framework for interactive proofs in separation logic. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi: 10.1145/3236772. URL https://doi.org/10.1145/3236772.

[22] Lars König. An improved interface for interactive proofs in separation logic, October 2022.

[23] leanprover-community. iris-lean: Lean 4 port of iris, a higher-order concurrent separation logic framework. https://github.com/leanprover-community/iris-lean/, 2026. Accessed: 18th Jan 2026.

[24] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013. ISBN 0262026651.